

Summative Assignment: WEBD2A Web Development 2

For Learner Use:

Surname of Learner	Damane
Name of Learner	Lolwethu
Learner ID	0503075385081
Student Number	LD24CPT053
Date of Test Given	08/04/2026

For Assessors Use:

Name of Facilitator	Amber Mannoe
Name of Assessor	Amber Mannoe
Mark Allocation	160
Mark Obtained	41
Competency Status (C / NYC)	NYC

Knowledge and Practical Outcomes Assessed	
118707 Occupational Certificate: Software Developer	➤ 251201-005-00-KM-06 Software Development with HTML5, Opensource Frameworks and Libraries, Level 5, 16 Credits. ➤ 251201-005-00-PM-08, Develop Software using HTML5, Opensource Frameworks and Libraries, Level 5, 16 Credits. ➤ 251201-005-00-PM-09 Design and Build Web Applications, Desktop Graphical User Interfaces or Mobile Apps, Level 5, 8 Credits.

Candidates Signature		Date of Submission	04/05/2026
Assessors Signature		Date Marked	16/5/2026

INSTRUCTIONS

1. Read questions carefully.
2. Answer all questions.
3. You will complete a number of practical and theoretical questions.
4. Place screenshots in the space provided.
5. Write Answers in the space provided.
6. Complete the cover page in full.
7. Plagiarism will not be tolerated
8. Ensure your work is formatted neatly and is readable by the Assessor at all times.
9. One complete assignment must be submitted in editable format.
10. Any attachments must be submitted with the original submission.

This Assignment is set up in of 4 sections:

- Task A: Design a Web Application
- Task B: Build a functional REST AP
- Task C: Build a functional Web Interface
- Task D: Testing a Web Application

All Sections are to be completed in full, according to the instructions given.

COMPETENCY RATINGS:

- Not Yet Competent (NYC) – Student achieves less than 60%
- Competent (C) – Student achieves 60% – 74%
- Competent with Merit (C/M) – Student achieves 75% - 84%
- Competent with Distinction (C/D) – Student achieves 85% - 100%

SCENARIO

You work as a freelance web developer, and a local small business has contacted you to make a request to build a web application for them.

They have asked you to make a to-do list web application so that their employees can better understand their tasks. Users should be able to manage tasks through the web application's interface. They need to be able to see a list of existing tasks, add new tasks, and delete selected tasks.

Read through all the instructions carefully and complete the tasks in the order given.

Task A – Design a Web Application (70)**41**

1. Prior to designing your website it is important that you understand exactly what is required of the web application. Firstly, you will need to interview the owner of the local business to gain the following information: (10)
 - the aim and functionality of the web application
 - any user requirements
 - implementation of the web application
 - timescales for completing the web application

INTERVIEW QUESTIONS AND ANSWERS

Interviewer: What is the main purpose of this website and how can this bring a positive impact to the business?

Interviewee: main purpose of the website is to provide a platform where users can create, manage, and track tasks efficiently.

Interviewer: What is the basic requirement for the user to insert for the website to function?

Interviewee: The user is required to input a task title and a description. These inputs are essential for creating a task in the system.

Interviewer: What technologies would you like to be implemented for the backend and database?

Interviewee: I would use Node.js because it is lightweight, fast, and well-suited for building REST APIs. For the database, I would use SQLite for simplicity in development and testing, or MySQL for a more scalable production environment.

Interviewer: What is the expectation due date for this project to be completed?

Interviewee: For this project is approximately 1 to 2 weeks, depending on the level of features required.

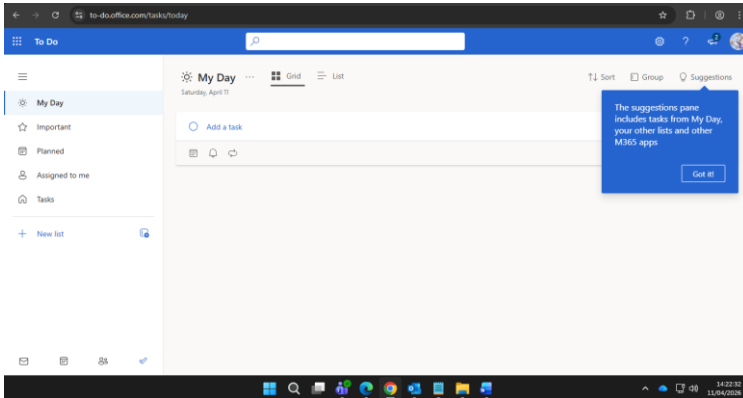
Interviewer: What would hero page motto would be with what colours and fonts?

Interviewee: Stay Productive. Stay Ahead ✓**10**

2. Based on the information that you have gathered from the owner of the local business, research **THREE** web applications that relate to what the owner requires. Include the following: (30)
 - a. Overview of the site
 - b. Screenshots
 - c. Pros and Cons (focus on User-Experience and the site objective)
 - d. Compare three web applications ¹
 - e. What you would adopt (why and how would it benefit your site and how can you make it better)
 - f. Draw a conclusion

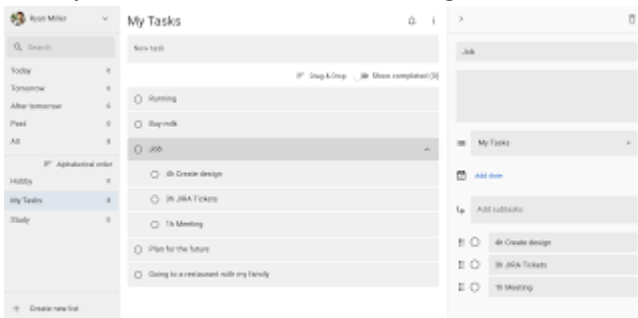
RESEARCH

Overview of the Microsoft To-Do is for users can be able to list their goals on what they need to achieve.
Microsoft 365 To-Do: <https://to-do.office.com/tasks/today>



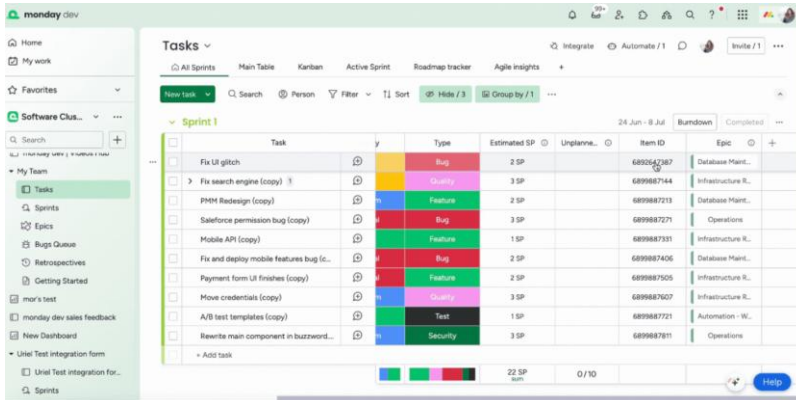
Pros	Cons
Shared lists and real-time updates support light collaboration.	Some users note sporadic bugs or missed alerts.
It is easy to organize tasks, add due dates, reminders, and break tasks into steps.	You can't color-code tasks and customization options are minimal.

Google Task is a task management application developed by Google and included with Google Workspace. Included initially as a feature in Gmail and Google Calendar is standalone app from 2018.



Pros	Cons
Completely free with no subscription required; available to anyone with a Google account	It is not suitable for team task management; you can't assign tasks to multiple users.
Cross devices are available on desktop, Android, iOS, and web, syncing across all devices.	Tasks don't easily sync with non-Google calendars or apps.

Monday.com are integrated and automated, built workflows, created templates, and expanded our community beyond our wildest expectations, we continued to grow. It is through this journey that the monday.com



Pros	Cons
Great for teams who need shared to-do lists, comments, file attachments, and updates.	Paid plans require a minimum of three users, which increases cost even if you're working alone
You can build your own to do system using templates, automations, and flexible structures.	Users report bugs, limited views, and inconsistent updates on the mobile version

D.

Microsoft To Do	Google Task	Monday.com
Allows users to quickly create and manage tasks without confusion	It keeps the interface clean and distraction-free.	Supports both individuals and teams with more advanced features
Improves productivity through reminders	Include progress tracking.	Improves task tracking with visual workflows

E. Improves user experience and encourages more frequent use of the site; visual organization helps users clearly track progress, and scalability allows the system to grow from individual use to team collaboration. Help users break down larger tasks into smaller, manageable parts.

F: j

✓20

- Once you have this information you will need to produce a short plan that outlines the details you have learned. Include use case(s) that defines how users will interact with the system

11(30)

IMPLEMENTATION PLAN (based on answers)

This website is a simple task manager where users can add, view, and delete their tasks. The main idea is to help users stay organized and keep track of what they need to do.

Language

I will use HTML, CSS, and JavaScript for the frontend, and Node.js with Express for the backend. The data will be stored using an SQLite database.

Core Features

Users will be able to create tasks by adding a title and description, view all their tasks, and delete tasks when they are done.

API

The system will use GET to fetch tasks, POST to add tasks, and DELETE to remove tasks.

✓8

USE CASE(S)**Create Task**

The user types in a title and description, then clicks add. The task is saved and shows up on the screen.

View Tasks

When the user opens the website, all saved tasks are displayed automatically.

Delete Task

The user clicks a delete button on a task, and it gets removed from the list. ✓3

Task B – Build a functional REST API (20)

For your web application to respond to user requests and store any data, you will need to build REST API to establish some form of communication between the server and client. You will also need to set up a connection to a SQL Database to store data for individual tasks. *

Create a SQL database that contains a table called **tasks** that is structured as follows:

(5)

COLUMN NAME	DATA TYPE
id	INTEGER (PRIMARY KEY)
title	TEXT
description	TEXT

P.S. Use **CREATE TABLE IF NOT EXISTS** at server startup to ensure the table gets created before the server begins accepting requests.

Your server should run on a suitable port that will not be in use by any other service or protocol.

Ensure your REST API supports the following requests:

1) GET / tasks

(5)

This GET request should respond with a list of all the tasks that exist in the database. In the request handler, you should be able to run a **SELECT** query on your database and respond with a list of all the tasks returned and a status code of **200**.

2) POST / tasks

(5)

This POST request should accept JSON data in the body of the request containing a title and a description of the task. In the request handler, you should be able to run an **INSERT** query on your data in to add a new task with the provided title and description. If everything works, your API should send a success response with a status code of **201**.

3) DELETE /tasks/:id

(5)

This DELETE request should accept a task ID as a request parameter in the URI. In the request handler, you should be able to run a **DELETE** query on your database to delete the task with the given ID. If everything works, your API should send a success response with a status code of **200**.

**The actual code will be assessed as well as the functional server. You need not host the server on a functional registered domain – it can be local.*

Task C – Build a functional Web Interface (50)

Now that your REST API is complete, it's time to build a web interface for your users. *

1) Write HTML and CSS code to build an interface for your users. (30)

Your web interface must feature the following:

- a list with a heading to display all available tasks
- a UI element to display the title and description of a single task
- a delete button on each task
- a form with 2 text inputs for a title and a description
- suitable CSS styling for each element

2) Write JavaScript code to communicate with the server using the REST API you built in Task B. (20)

Your code needs to be able to do the following:

- fetch the tasks data from the server using fetch
- dynamically render the tasks list from tasks data received
- delete selected tasks when the delete button is clicked
- add new tasks using the information entered in the form
- display user-friendly error messages when errors occur

****The actual code will be assessed as well as the functional web application. You need not host the web application on a functional registered domain – it can be local.***

Task D – Testing a Web Application (20)

You've finished your Web Application! Now it's time to test it to make sure it's suitable for users.

- 1) Test your REST API. Make a test log comparing the expected results and actual results of running each request. Provide screenshots as evidence for all testing done. (5)

- 2) Test your Web Application's user interface. Make a test log comparing expected results to actual results of testing each piece of functionality in the interface. In addition, test your web application in at least 2 different browsers. Provide screenshots as evidence for all testing done. (15)

Please reference the files and folders attached to this document below.